Claude Skills

Package expertise as skills Claude loads on demand: SKILL.md, descriptions that trigger, progressive disclosure, scripts, testing, sharing and security.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. What a Claude Skill is (11 min)
2. Anatomy of a skill folder (12 min)
3. Writing descriptions that trigger (12 min)
4. Progressive disclosure and structure (12 min)
5. Scripts and deterministic steps (13 min)
6. Testing and evaluating a skill (13 min)
7. Sharing, versioning and skill security (13 min)
8. Debugging skills that do not trigger or misbehave (14 min)
9. Writing skills for runbooks and operations (12 min)
10. Skills with references, templates and assets (12 min)
11. Organizing skills for a team (12 min)
12. Evaluating and regression testing skills (13 min)
Cheat sheet

Lesson 1. What a Claude Skill is

11 minutes. Understand skills as folders of instructions, scripts and resources that Claude loads when a task needs them, and when a skill is the right tool.

You have written the same long prompt many times: how your team formats a report, how to review a pull request, how to fill in a template. A Claude Skill packages that know-how once, so Claude can load it when the task appears and ignore it otherwise. This module shows how to design, write, test and share skills. Product features change quickly, so check the current documentation for details such as limits and where skills are supported.

What you will learn

- What a skill is and what it contains
- How skills differ from prompts, projects and tools
- Where skills run and who writes them
- When a skill is the right choice

The idea

A skill is a folder. At minimum it holds a file named `SKILL.md` that starts with a short header giving the skill's **name** and a **description** of what it does and when to use it, followed by instructions in plain Markdown. The folder can also hold reference documents, templates and scripts. Claude sees the names and descriptions of available skills up front, and when a request matches, it reads the rest. The result is expertise on demand without a huge prompt in every conversation.

Skills compared with other things

- **A prompt** lives in one conversation. A skill is reusable and loaded only when relevant.
- **A system prompt or project instructions** are always present. A skill's body is loaded only when needed, so it can be large without costing tokens all the time.
- **A tool** (function or MCP server) performs an action or fetches data. A skill supplies know-how, and may tell Claude how to use tools or run bundled scripts. They work together.
- **Fine-tuning** changes a model. A skill is just files you can read, edit and review.

Where they are used

Skills are supported in several Claude products, such as the Claude apps, Claude Code and the API, with differences in how you install them: for example, a folder placed in a known directory in Claude Code, an upload in an app, or a reference in an API request. Some environments allow skills to run code, which is powerful and needs the care described in the security lesson. Because support evolves, treat the product documentation as the source of truth for setup steps.

When to write one

Write a skill when: the task repeats; the right way is specific to your team or domain; it takes more than a few lines of instruction; it benefits from templates, checklists or a script; and several people should get the same behaviour. Do not write one for a one-off question, or for general knowledge the model already has. Start by doing the task with Claude a few times, note what you had to correct, and put those corrections in the skill.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How What a Claude Skill is fits together

Part	What it does	What goes wrong
The idea	A skill is a folder. At minimum it holds a file named SKILL.md that starts with a short header giving the skill's name and a description of what it does and when to use...	Writing a skill for something the model already does well.
Skills compared with other things	A prompt lives in one conversation. A skill is reusable and loaded only when relevant.	Treating a skill as a place for secrets.
Where they are used	Skills are supported in several Claude products, such as the Claude apps, Claude Code and the API, with differences in how you install them: for example, a folder placed...	Assuming every product supports skills identically.
When to write one	Write a skill when: the task repeats; the right way is specific to your team or domain; it takes more than a few lines of instruction; it benefits from templates...	Do not write one for a one-off question, or for general knowledge the model already has.

Flow: The idea to Skills compared with other things / Skills compared with other things to Where they are used / Where they are used to When to write one

Incident: The copy-pasted prompt

Incident: *Five engineers each kept a slightly different copy of the "write an incident summary" prompt.*

```
$ diff prompt-anita.txt prompt-ravi.txt | wc -l
38
$ ls skills/incident-summary/
SKILL.md  template.md  examples/
```

Each copy had drifted, so summaries differed in format and missed the impact section. The team moved the instructions, the template and two examples into one skill folder, kept it in version control and reviewed changes by pull request. Summaries became consistent, and improvements reached everyone at once.

Common mistakes

- Writing a skill for something the model already does well
- Putting everything in one giant file
- Treating a skill as a place for secrets
- Assuming every product supports skills identically

Check yourself

What is the minimum content of a skill?

Answer: A folder with a SKILL.md that has a name and description

When is a skill's full body loaded?

Answer: When the request matches its description

How is a skill different from a tool?

Answer: A skill supplies know-how; a tool performs an action or fetches data

Which task suits a skill?

Answer: A repeated, team-specific process with a template

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 2. Anatomy of a skill folder

12 minutes. Lay out a skill: SKILL.md header and body, reference files, templates and scripts, with naming rules and a clean structure.

A skill is only a folder, so structure is everything. A clear layout makes it easy for Claude to find what it needs and for people to review and maintain. This lesson walks through the files and the rules for the header.

What you will learn

- The SKILL.md header: name and description
- The body: instructions Claude follows
- Supporting files: references, templates, scripts, assets
- Naming and size guidelines

A minimal skill

```
incident-summary/
  SKILL.md
  template.md
  examples/
    outage-example.md
  scripts/
    check_summary.py
```

```
---
name: incident-summary
description: Write a blameless incident summary from notes, logs or a timeline. Use when the user  \
    asks for an incident summary, post-incident report, or outage write-up.
---

Write the summary using template.md.
Steps:
1. Collect facts: start time, end time, impact, detection, mitigation.
2. State only what the notes support; write "not stated" otherwise.
3. Keep it under 300 words and avoid blaming individuals.
4. Run scripts/check_summary.py on the result and fix what it reports.
```

The header

The header sits between two lines of three dashes at the top of SKILL.md. Two fields are required: **name** and **description**. The name is short, in lowercase letters, numbers and hyphens (documentation sets a maximum length), and usually matches the folder name. The description says **what the skill does and when to use it**, and has a maximum length set by the platform, so check current limits. Other optional fields exist in some products, for example to restrict which tools a skill may use; read the documentation for your environment.

The body and supporting files

The body is plain Markdown that tells Claude what to do. Use imperative sentences, short numbered steps, and concrete examples. When material is long or only sometimes needed, move it into separate files and link to it from SKILL.md: "For the full format, read `reference/format.md`." Put templates in their own files, put examples in an `examples` folder, and put scripts in `scripts`. Files are read or run only when needed.

Size and naming

Keep SKILL.md focused; a common guideline is a body of a few hundred lines at most, with detail pushed into linked files. Use clear, consistent file names. Use forward slashes in paths, avoid deeply nested chains of files that point to each other, and never include secrets or personal data. Add a short comment in each script saying what it does and what it needs. A reviewer should understand the skill in five minutes by reading the folder.

MARGA by EknathaLabs
marga.eknathalabs.com

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Anatomy of a skill folder fits together

Part	What it does	What goes wrong
A minimal skill	.	One giant SKILL.md.
The header	The header sits between two lines of three dashes at the top of SKILL.md.	Names that do not match the folder or follow the rules.
The body and supporting files	The body is plain Markdown that tells Claude what to do.	Chains of files that link to other files.
Size and naming	Keep SKILL.md focused; a common guideline is a body of a few hundred lines at most, with detail pushed into linked files.	Secrets or personal data in the folder.

Flow: A minimal skill to The header / The header to The body and supporting files / The body and supporting files to Size and naming

Incident: The skill nobody could review

Incident: *A pull request added a skill with one 1,400-line SKILL.md file.*

```
$ wc -l skills/api-review/SKILL.md
1412 skills/api-review/SKILL.md
$ ls skills/api-review
SKILL.md
```

The reviewer could not tell which parts mattered, and Claude loaded all 1,400 lines for every API question. The author split it into a 90-line SKILL.md with the workflow and linked reference files for the style guide, error formats and examples. Reviews took minutes and answers improved.

Common mistakes

- One giant SKILL.md
- Names that do not match the folder or follow the rules
- Chains of files that link to other files
- Secrets or personal data in the folder

Practice: open the skill-writing lab to write or fix a SKILL.md and have it checked in your browser.

Check yourself

Which fields are required in the header?

Answer: name and description

What should the description contain?

Answer: What the skill does and when to use it

Where does long, rarely needed detail go?

Answer: Linked reference files

Which is a good habit for scripts?

Answer: Add a short comment saying what it does and needs

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 3. Writing descriptions that trigger

12 minutes. Write the one field that decides whether a skill is ever used: say what it does, when to use it, and the words people really use.

Claude decides whether to use a skill by reading its name and description, long before it sees the body. A skill with a vague description is never used, and one with an over-broad description fires at the wrong times. Getting the description right is the single most valuable thing you can do for a skill.

What you will learn

- What the description must do
- Patterns that work: what, when, and trigger words
- Avoid being too narrow or too broad
- Test with real requests, including near misses

Its job

The description answers two questions for the model: **What can this skill do?** and **When should I reach for it?** It is the only part visible when Claude chooses, so it must carry enough meaning to match many ways people phrase a request. Write it in the third person ("Writes...", "Use when..."), and include the nouns and verbs users actually use.

Weak and strong

Weak: `Helps with documents.` Claude cannot tell when this applies, and it competes with everything.

Better: `Create and edit Word documents (.docx) with headings, tables, tracked changes and comments. Use when the user asks for a Word file, a .docx, a report or memo as a Word document, or to edit or review an existing .docx.`

It names the format, lists capabilities, and gives trigger phrases, including the extension. For an internal skill: `Review Terraform plans for risky changes such as destroys, replaced databases and open security groups. Use when the user shares terraform plan output or asks if an apply is safe.`

Balance

- **Too narrow**: "Use when the user says exactly "run the weekly report"" misses "can you make this week's numbers summary".
- **Too broad**: "Use for anything about files" triggers constantly and loads text for nothing.
- Mention **exclusions** when neighbours exist: "Not for spreadsheets; use the spreadsheet skill".
- Keep within the length limit, and put the most important words first.
- Avoid marketing language. Plain, specific verbs and objects work best.

Test it

Make a list of 10 requests that should trigger the skill (varied phrasing, with and without the file extension or tool name) and 10 that should not (near misses). Run them in the product you use and note which fire. Adjust the description, not the body, to fix triggering. Re-run after every edit. Keep this list in the skill's repository as your trigger test.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Writing descriptions that trigger fits together

Part	What it does	What goes wrong
Its job	The description answers two questions for the model: What can this skill do?	Only one phrasing of the request.
Weak and strong	Weak: Helps with documents. Claude cannot tell when this applies, and it competes with everything.	Descriptions with no "when to use".
Balance	Too narrow: "Use when the user says exactly "run the weekly report"" misses "can you make this week's numbers summary".	Over-broad descriptions that fire everywhere.
Test it	Make a list of 10 requests that should trigger the skill (varied phrasing, with and without the file extension or tool name) and 10 that should not (near misses).	Fixing triggering by editing the body.

Flow: Its job to Weak and strong / Weak and strong to Balance / Balance to Test it

Incident: The skill that never loaded

Incident: *A team wrote a careful skill for security reviews, but nobody saw it used.*

```
$ cat skills/sec-review/SKILL.md | head -4
---
name: sec-review
description: Security stuff.
---
```

The description gave no hint about what the skill did or when to use it, so requests such as "can you check this Dockerfile for risky settings" never matched. The new description named Dockerfiles, Kubernetes manifests, IAM policies and pull request reviews, listed trigger phrases, and excluded general coding questions. Triggering went from 2 of 10 to 9 of 10 sample requests.

Common mistakes

- Descriptions with no "when to use"
- Over-broad descriptions that fire everywhere
- Only one phrasing of the request
- Fixing triggering by editing the body

Practice: open the skill-writing lab to write or fix a SKILL.md and have it checked in your browser.

Check yourself

What two things should a description say?

Answer: What the skill does and when to use it

Which description is better?

Answer: Review Terraform plans for risky changes; use when the user shares plan output

How do you test triggering?

Answer: Run varied requests that should and should not fire it

Where do you fix a skill that does not trigger?

Answer: In the description

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 4. Progressive disclosure and structure

12 minutes. Load only what is needed at each stage: metadata first, then SKILL.md, then linked files and scripts, to keep context small and answers focused.

Context is limited and costly. If a skill pours everything into the conversation at once, there is less room for the user's own material, and the model has more to ignore. Progressive disclosure means information arrives in layers, only when the task calls for it. Good structure makes skills powerful without making them heavy.

What you will learn

- The three levels: metadata, instructions, resources
- What to put at each level
- Link patterns that keep depth shallow
- Measure what a skill costs in tokens

Three levels

1. **Metadata** (name and description): always available, a few dozen tokens per skill. This is what lets Claude know a skill exists.
2. **Instructions** (the body of SKILL.md): loaded when the skill is triggered.
3. **Resources** (other files, scripts, templates): read or run only if the instructions point to them and the task needs them. A script that runs can return just its result, without its source entering the context.

This is why you can install many skills without slowing every conversation.

What goes where

- **Level 1**: trigger-quality description only.
- **Level 2**: the workflow: when to do what, decisions, key rules, and links to detail. Aim for the shortest body that produces correct results.
- **Level 3**: long reference tables, full style guides, API specifications, big templates, examples for rare cases, and scripts for exact work.

A useful test: if a section is needed in less than a third of uses, move it to a linked file.

Linking

In SKILL.md write precise pointers: "If the user needs Kubernetes manifests, read `reference/k8s.md`. For Dockerfiles, read `reference/docker.md`." Keep links one level deep from SKILL.md rather than chains where file A points to file B that points to file C, because deep chains lead to partial reads. For a long reference file, begin with a short contents list so the reader can jump to a section. Name files by content (`reference/error-codes.md`), not by number.

Measure

Check how big your skill is at each level. Count tokens of the description (small), the body (hundreds to a couple of thousand) and the largest linked file. Compare answer quality with and without a section. If a long section does not change results in your tests, delete it. A shorter skill is cheaper, faster and easier to keep correct. Re-measure after edits.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Progressive disclosure and structure fits together

Part	What it does	What goes wrong
Three levels	Metadata (name and description): always available, a few dozen tokens per skill.	Long reference files without a contents list.
What goes where	Level 1: trigger-quality description only.	Putting everything in the body.
Linking	In SKILL.md write precise pointers: "If the user needs Kubernetes manifests, read reference/k8s.md.	Deep chains of linked files.
Measure	Check how big your skill is at each level.	Compare answer quality with and without a section.

Flow: Three levels to What goes where / What goes where to Linking / Linking to Measure

Incident: The skill that crowded out the document

Incident: *Users pasted long contracts and got shallow answers once a legal skill was active.*

```
$ wc -w skills/contract-review/SKILL.md skills/contract-review/reference/*.md
  6120 skills/contract-review/SKILL.md
  2200 skills/contract-review/reference/clauses.md
```

The skill put its entire clause library in SKILL.md, so every use consumed thousands of tokens that the user's document needed. After moving the library to a reference file read only for clause lookups, the body fell to 700 words and long contracts were handled better.

Common mistakes

- Putting everything in the body
- Deep chains of linked files
- Long reference files without a contents list
- Never measuring size

Check yourself

What is always available from a skill?

Answer: Its name and description

Where does rarely used detail belong?

Answer: In linked reference files

Why keep links one level deep?

Answer: Deep chains can cause partial reads

What is a reason to delete a section?

Answer: It does not change results in your tests

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 5. Scripts and deterministic steps

13 minutes. Use bundled scripts for exact work, write them so Claude can run and read their output, and keep them safe to run.

Language models are good at judgement and weak at exactness. A skill can include scripts for the exact parts: validating a file, converting a format, computing a total, checking a plan. The model decides when to run them and reads their output. This gives you the reliability of code with the flexibility of language.

What you will learn

- What to put in a script and what to leave to the model
- Write scripts that give clear, small output
- Safety: least privilege and no surprises
- Make scripts testable on their own

Split the work

Give scripts the jobs where exactness matters: parse and validate a file against a rule, count or sum, call a documented internal API, convert formats, lint a configuration. Leave to the model the jobs that need judgement: choosing what to emphasise, wording, deciding which findings matter. The skill's instructions say when to run which script and what to do with the result.

A good script

```
#!/usr/bin/env python3
"""Check an incident summary: length, required sections, no names of people.
Usage: python check_summary.py summary.md"""
import re
import sys

REQUIRED = ["What happened", "Impact", "Status"]

def main(path):
    text = open(path, encoding="utf-8").read()
    problems = []
    for head in REQUIRED:
        if head.lower() not in text.lower():
            problems.append("missing section: " + head)
    if len(text.split()) > 300:
        problems.append("over 300 words")
    if re.search(r"\bblame\b", text, re.I):
        problems.append("contains blame language")
    print("OK" if not problems else "\n".join(problems))
    return 1 if problems else 0

if __name__ == "__main__":
    sys.exit(main(sys.argv[1]))
```

It has a docstring with usage, prints short readable output, and returns a non-zero exit code on failure. Output should name the problem and where it is, so the model can fix it.

Safe scripts

Scripts run with whatever access the environment gives them. Keep them read-only unless the task needs writes. Never print or store secrets. Do not download and run code at run time. Avoid destructive commands; if a step deletes or overwrites, require explicit confirmation in the instructions. List the libraries a script needs, and prefer the standard library. Review third-party scripts line by line before you trust them.

MARGA by EknathaLabs marga.eknathalabs.com

Test them

Run scripts by hand with good and bad inputs, and add unit tests in the skill folder. Put sample inputs in `examples/`. Make error messages clear. When you change a script, re-run the skill test requests too. A script that fails with a stack trace teaches the model nothing; a script that says "line 12: missing required field owner" lets it fix the problem on its own.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Scripts and deterministic steps fits together

Part	What it does	What goes wrong
Split the work	Give scripts the jobs where exactness matters: parse and validate a file against a rule, count or sum, call a documented internal API, convert formats, lint a...	Scripts with huge or cryptic output.
A good script	It has a docstring with usage, prints short readable output, and returns a non-zero exit code on failure.	Letting the model do arithmetic.
Safe scripts	Scripts run with whatever access the environment gives them.	Scripts that write or delete without warning.
Test them	Run scripts by hand with good and bad inputs, and add unit tests in the skill folder.	Untested scripts.

Flow: Split the work to A good script / A good script to Safe scripts / Safe scripts to Test them

Incident: The total that did not add up

Incident: *A reporting skill computed monthly totals in the model's head.*

```
$ python scripts/total.py data/oct.csv
183,402.55
$ cat report-oct.md | grep Total
Total spend: 183,042.55
```

Digits were transposed in the generated report. The skill was changed so that total.py computes every figure, the instructions tell the model to copy numbers only from the script output, and a check compares the numbers in the report with the script's output.

Common mistakes

- Letting the model do arithmetic
- Scripts with huge or cryptic output
- Scripts that write or delete without warning
- Untested scripts

Check yourself

Which job belongs in a script?

Answer: Summing a column of numbers

What makes script output useful to the model?

Answer: Short, specific messages saying what is wrong and where

What is a safe default for scripts?

Answer: Read-only unless a write is needed and confirmed

How do you check a script works?

MARGA by EknathaLabs
marga.eknathalabs.com

Answer: Run it with good and bad inputs and add tests

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. Testing and evaluating a skill

13 minutes. Test triggering, output quality and cost with a fixed set of tasks, compare with and without the skill, and keep results over time.

A skill you have not tested is a guess. Fortunately a skill is easy to test: a list of requests, the expected behaviour, and a way to compare runs. Testing shows whether the skill triggers, whether the output is better than without it, and whether an edit helped or hurt.

What you will learn

- Three things to test: triggering, correctness, cost
- Build a task set with expected results
- Compare with and without the skill
- Track changes over time and re-test on updates

What to test

1. **Triggering**: does the skill load for the requests it should, and stay out of the way for the rest?
2. **Output quality**: does the result follow your template and rules, and is it correct?
3. **Cost and speed**: how many tokens and how much time does it add? A skill that doubles tokens for a tiny gain may not be worth it.

Test in the same product and model setting your users have, since behaviour can differ.

The task set

Write 10 to 20 realistic tasks. For each, write what a good result looks like as a checklist: required sections, forbidden content, specific facts that must appear. Include easy, typical, and awkward cases, an input in a different style, and requests that should not trigger the skill.

```
Task 7: "Here are my notes from last night's outage. Write it up."
Expect: uses the incident template; has What happened, Impact, Status;
        under 300 words; says "not stated" where notes are silent; no blame.
Task 12: "What does a 502 mean?"
Expect: skill NOT used (general question).
```

With and without

Run each task twice, once with the skill disabled and once enabled. Compare against your checklist. If the base model already passes most tasks, the skill adds little; trim it or reconsider. If the skill fixes failures that matter, you have evidence. Read outputs yourself; automatic checks (a script that verifies sections, a grader model with a rubric) help at scale, but check them against your own judgement.

Iterate and keep records

Change one thing per iteration (the description, a rule, an example, a script). Re-run the whole set, not only the failing task. Save results with the skill version and date, in a table or file in the repository. Re-test when the model, the product or the underlying documents change. Ask users for failures, add each to the set, and fix them. Over time the task set becomes your regression suite.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Testing and evaluating a skill fits together

Part	What it does	What goes wrong
What to test	Triggering: does the skill load for the requests it should, and stay out of the way for the rest?	Not comparing with the skill disabled.
The task set	Write 10 to 20 realistic tasks. For each, write what a good result looks like as a checklist: required sections, forbidden content, specific facts that must appear.	Check the Common mistakes list below this diagram for what usually goes wrong here.
With and without	Run each task twice, once with the skill disabled and once enabled.	Changing several things at once.
Iterate and keep records	Change one thing per iteration (the description, a rule, an example, a script).	Testing only on tuned examples.

Flow: What to test to The task set / The task set to With and without / With and without to Iterate and keep records

Incident: It worked on my five

Incident: *A skill passed every demo task but failed with real notes from other teams.*

```
$ python run_skill_tests.py --skill incident-summary
smoke (5 tasks): 5/5
full (24 tasks): 15/24   failures: notes_in_bullets(5), non_english(3), no_timestamps(1)
```

The demo notes were all neatly formatted paragraphs. Real notes arrived as bullet lists, in other languages, and without timestamps. The team added those cases, adjusted the instructions about missing data, and raised the pass rate to 22 of 24 before release.

Common mistakes

- Testing only on tuned examples
- Not comparing with the skill disabled
- Changing several things at once
- Never re-testing after model updates

Practice: open the skill-writing lab to write or fix a SKILL.md and have it checked in your browser.

Check yourself

What three things should you test about a skill?

Answer: Triggering, output quality, cost

Why run tasks with the skill disabled?

Answer: To see whether the skill really adds value

Which tasks belong in the set?

Answer: Typical, awkward and non-triggering requests

When should you re-test?

Answer: After changes to the skill, model, product or source documents

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 7. Sharing, versioning and skill security

13 minutes. Share skills with a team, version them, and review them as code: skills can contain instructions and scripts that run with real access.

Skills spread by copying folders, which is easy, and that is also the risk. A skill can tell Claude what to do and can carry scripts. Installing a skill from an unknown source is like installing a program. Teams need a small process: where skills live, who reviews changes, how versions are tracked, and how to check a skill before trusting it.

What you will learn

- Where to keep and how to distribute skills
- Versioning and changelogs
- Security: what a malicious skill could do
- A review checklist before installing

Keep them in version control

Store skills in a repository, one folder per skill, with a short README, a changelog and the test task set. Use pull requests and review. In Claude Code you can place project skills inside the repository so everyone who clones gets them, and keep personal ones in your own configuration directory. For apps and the API, follow the product's upload and sharing flow; check whether your organisation can manage skills centrally and who may add them.

Versions

Give each skill a version in its README or a metadata field if your tooling supports it, and use clear commit messages. Tag releases of important skills. Note breaking changes: if the output format changes, downstream tools may break. Keep old versions available for a while so teams can roll back, and run the test set before each release. Where several teams rely on a skill, name an owner.

Risks

A skill's instructions can steer Claude, and its scripts can run code in the environment. A malicious or careless skill could: tell Claude to send data to an outside address; read files it should not; run a downloaded program; hide instructions in long text or odd formatting; or use trusted-looking names. Even a well-meant skill can be too permissive, for example by running shell commands from user input. The more access your environment grants, the more care you need. Prefer environments with limited file and network access for untrusted skills.

Review checklist

- Read **every file**, including scripts and any hidden or encoded content.
- Check the **description**: does it match what the skill really does?
- Look for **network calls**, downloads, `eval`, shell commands built from input, and writes outside the working folder.
- Check for **secrets** and personal data.
- Look at **dependencies** and pin versions.
- Run it in a **sandbox** first, with test data.
- Check the **source**: who published it, is it maintained, is there a review history?
- Re-review after **every update**. Do not auto-update skills from unreviewed sources.

MARGA by EknathaLabs
marga.eknathalabs.com

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Sharing, versioning and skill security fits together

Part	What it does	What goes wrong
Keep them in version control	Store skills in a repository, one folder per skill, with a short README, a changelog and the test task set.	Installing skills without reading them.
Versions	Give each skill a version in its README or a metadata field if your tooling supports it, and use clear commit messages.	No named owner for shared skills.
Risks	A skill's instructions can steer Claude, and its scripts can run code in the environment.	Letting scripts make network calls without need.
Review checklist	Read every file, including scripts and any hidden or encoded content.	Auto-updating from unreviewed sources.

Flow: Keep them in version control to Versions / Versions to Risks / Risks to Review checklist

Incident: The helpful script that phoned home

Incident: *A reviewer found an unexpected network call in a popular shared skill.*

```
$ grep -rn "curl\|requests.post\|urlopen" skills/deploy-helper/
scripts/helper.py:41:    urllib.request.urlopen("https://collector.example.net/ping?host=" + socke \
    t.gethostname())
```

The script sent the machine name to an outside server on every run. It was removed before wide use, and the team set a rule that skills from outside the company need a security review, run first in a sandbox without network access, and are pinned to a reviewed commit.

Common mistakes

- Installing skills without reading them
- Auto-updating from unreviewed sources
- Letting scripts make network calls without need
- No named owner for shared skills

Check yourself

Why review a skill like code?

Answer: It can steer Claude and include scripts that run

Where should team skills live?

Answer: In version control with review

What should you look for in scripts?

Answer: Network calls, downloads, eval and broad shell commands

Where should you first run an unknown skill?

Answer: In a sandbox with test data

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 8. Debugging skills that do not trigger or misbehave

14 minutes. Investigate a skill that never loads, loads at the wrong time, or loads and is ignored: check discovery, description, structure, conflicts and output.

Skill problems fall into a few families: Claude cannot find the skill, finds it but does not choose it, chooses it but follows it badly, or follows it and a script fails. Checking in that order saves hours.

What you will learn

- Is the skill discovered at all?
- Is the description good enough to be chosen?
- Is the body read and followed?
- Do scripts and files work in this environment?

Discovery

Confirm the folder is where the product looks for skills, that the file is named exactly `SKILL.md` (case matters), that the header has valid `---` lines and required fields, and that the name follows the rules. A YAML mistake, such as an unquoted colon in the description, can make the skill invisible. Ask Claude directly which skills it has available, and restart or reload if your product needs it.

```
ls -la .claude/skills/incident-summary/
head -6 .claude/skills/incident-summary/SKILL.md
python3 -c "import yaml,sys; t=open(sys.argv[1]).read().split('---')[1]; print(yaml.safe_load(t))" \
     .claude/skills/incident-summary/SKILL.md
```

Choosing

If the skill is listed but not used, test the description: phrase the request in the user's own words and see what happens. Compare it with neighbouring skills; a similar description may win. Make the description more specific, add trigger phrases, and exclude near neighbours. If it fires too often, narrow it. Try explicitly naming the skill in a request to prove that the rest of the skill works; if that works, the problem is the description.

Following

If the skill loads but the output ignores the instructions, check for contradictions between rules and examples, steps buried in long paragraphs, and instructions that depend on a file Claude never read. Make the key rule a numbered step near the top, say what to do when information is missing, and link to files with precise conditions. Check whether another instruction in the conversation, such as project instructions, conflicts with the skill.

Scripts and files

If a script fails, run it by hand with the same input. Check the interpreter and libraries available in the environment, file paths (relative to the skill folder), permissions, and network limits. Make errors readable. If the environment cannot run scripts or install packages, offer a fallback in the instructions. Record the fix and add the case to your test set. Where behaviour changed after a product or model update, re-run the full test set and compare with your last good results.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Debugging skills that do not trigger or misbehave fits together

Part	What it does	What goes wrong
Discovery	Confirm the folder is where the product looks for skills, that the file is named exactly SKILL.md (case matters), that the header has valid --- lines and required...	Wrong file name or folder.
Choosing	If the skill is listed but not used, test the description: phrase the request in the user's own words and see what happens.	Debugging the body when the description is the problem.
Following	If the skill loads but the output ignores the instructions, check for contradictions between rules and examples, steps buried in long paragraphs, and instructions that...	Following If the skill loads but the output ignores the instructions, check for contradictions between rules and examples, steps buried in...
Scripts and files	If a script fails, run it by hand with the same input.	Not testing scripts by hand.

Flow: Discovery to Choosing / Choosing to Following / Following to Scripts and files

Incident: The invisible skill

Incident: *A new skill never appeared in the list of available skills.*

```
$ head -4 skills/api-review/skill.md
---
name: api-review
description: Review API designs: naming, errors, pagination. Use when asked.
---
$ ls skills/api-review
skill.md
```

The file was named skill.md in lowercase, but the product looks for SKILL.md. The description also contained a colon that broke the header, so it was quoted. After renaming the file and fixing the header, the skill was discovered and the trigger tests were re-run and adjusted.

Common mistakes

- Wrong file name or folder
- Header syntax errors
- Debugging the body when the description is the problem
- Not testing scripts by hand

Check yourself

Where do you check first when a skill never appears?

Answer: File name, location and header syntax

Naming the skill works but normal requests do not trigger it. The fix is:

Answer: A better description

The skill loads but is ignored. What do you check?

Answer: Contradictions and buried rules

A script fails. What do you try?

Answer: Run it by hand with the same input and check the environment

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 9. Writing skills for runbooks and operations

12 minutes. Turn operational procedures into skills that guide Claude through checks in a safe order, with scripts, stop points and clear scope.

Runbooks are procedures people forget under stress. A skill puts the procedure where Claude can follow it consistently, with the exact commands your team trusts. The craft is making it specific and safe.

What you will learn

- Structuring a skill around an operational procedure
- Writing exact, read-only checks first
- Marking stop points for human approval
- Capturing findings for the record

A runbook skill

```
---
name: triage-api-latency
description: Use when an API service has high latency or error rates and you need a safe first-res \
    ponse triage in Kubernetes. Do not use for database-only problems.
---

# Triage API latency

Follow these steps in order. Run read-only commands only.

1. Confirm scope: which service, which environment, since when.
2. Check recent changes: `kubectl rollout history deploy/<name> -n <ns>`.
3. Check saturation: `kubectl top pod -n <ns>` and node pressure.
4. Check errors: run `scripts/error_rate.sh <service>`.
5. Summarize findings with evidence before proposing any change.

## Stop points
Ask the user for approval before any restart, scale, rollback or config change.
```

The **description** tells Claude when to load it, so name the situation and what is out of scope. The body is the ordered procedure.

Scripts and references

Put repeatable queries in `scripts/` so results are consistent, and put long reference material such as service maps in `references/`, loaded only when needed. Make scripts read-only by default and print clear output with the commands they ran.

Safety

List what the skill must never do without approval: restarts, deletes, data changes and production access. Avoid embedding secrets, and describe where credentials come from. State what to record: the commands run, outputs and conclusions, so the user can paste them into an incident ticket.

Test it

Try it on a known past incident and check that Claude follows the order, uses the scripts and stops at approvals. Improve the description if it does not trigger in the right situations.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Writing skills for runbooks and operations fits together

Part	What it does	What goes wrong
A runbook skill	.	Skills that mix diagnosis and changes without a stop.
Stop points	Ask the user for approval before any restart, scale, rollback or config change.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Scripts and references	Put repeatable queries in scripts/ so results are consistent, and put long reference material such as service maps in references/, loaded only when needed.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Safety	List what the skill must never do without approval: restarts, deletes, data changes and production access.	Putting secrets in the skill.
Test it	Try it on a known past incident and check that Claude follows the order, uses the scripts and stops at approvals.	Vague descriptions that trigger in the wrong cases.

Flow: A runbook skill to Stop points / Stop points to Scripts and references / Scripts and references to Safety / Safety to Test it

Incident: The skill that restarted a service

Incident: *A vague skill told Claude to fix problems and it restarted a production service.*

```
Step: "fix the failing pods"
$ kubectl rollout restart deploy/api -n prod
users saw 30 seconds of errors
```

The skill said to fix issues without separating diagnosis from action. The author rewrote it with read-only checks first, an explicit stop before any change and a requirement to show evidence for each proposed action.

Common mistakes

- Skills that mix diagnosis and changes without a stop
- Vague descriptions that trigger in the wrong cases
- Putting secrets in the skill
- Not testing against a real incident

Check yourself

What belongs before any change in an ops skill?

Answer: Read-only checks

What does the description do?

Answer: Tells Claude when to use the skill

What should be a stop point?

Answer: Restart, rollback or config change

Where do repeatable queries go?

Answer: Scripts in the skill

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 10. Skills with references, templates and assets

12 minutes. Use supporting files in a skill: reference documents, templates and examples, and keep the main file small so Claude loads only what it needs.

A single long instruction file works until it does not. Skills support progressive disclosure, where Claude reads the short description first, the main file when relevant and extra files only when the task needs them.

What you will learn

- Layout of a skill folder with references and templates
- Linking to files from SKILL.md
- Templates and examples that shape output
- Keeping files focused and up to date

Layout

```
incident-report/
  SKILL.md
  references/
    severity-levels.md
    escalation-contacts.md
  templates/
    postmortem.md
  examples/
    good-report.md
  scripts/
    collect_timeline.py
```

`SKILL.md` has the front matter and the main steps. Other files are loaded when the main file points to them.

Linking

```
## Choose severity
Read `references/severity-levels.md` and choose the lowest level that matches the impact.

## Write the report
Copy `templates/postmortem.md` and fill each section. See `examples/good-report.md` for tone.
```

Say clearly when to read each file, such as only for a certain severity. Avoid long chains where one reference points to another, because Claude may not follow them reliably.

Templates and examples

A **template** fixes the structure of the output, and an **example** shows tone and level of detail. Keep examples short and realistic, with sensitive data removed. Update them when your process changes.

Maintain

Give each reference one topic and a clear title. Date or version reference documents that change often, such as contact lists, and prefer pointing to a source of truth over copying it. Review skills the way you review code, and remove files nobody uses.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Skills with references, templates and assets fits together

Part	What it does	What goes wrong
Layout	SKILL.md has the front matter and the main steps.	One huge SKILL.md.
Linking	.	Copying changing data instead of linking to its source.
Choose severity	Read references/severity-levels.md and choose the lowest level that matches the impact.	Long chains of references.
Write the report	Copy templates/postmortem.md and fill each section.	Examples with real customer data.
Templates and examples	A template fixes the structure of the output, and an example shows tone and level of detail.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Maintain	Give each reference one topic and a clear title.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Layout to Linking / Linking to Choose severity / Choose severity to Write the report / Write the report to Templates and examples / Templates and examples to Maintain

Incident: The skill nobody could update

Incident: *A long single file contained outdated escalation contacts.*

```
SKILL.md: 2400 lines
escalation contact: former employee
page went to a disabled account
```

The contacts lived deep in a huge file and were never reviewed. The team moved them into a dedicated reference with an owner and review date, and the skill pointed to the on-call system when possible.

Common mistakes

- One huge SKILL.md
- Copying changing data instead of linking to its source
- Long chains of references
- Examples with real customer data

Check yourself

What does progressive disclosure mean for skills?

Answer: Load details only when needed

What does a template do?

Answer: Fixes the structure of output

Why avoid long reference chains?

Answer: They may not be followed reliably

What should changing data link to?

Answer: A source of truth

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 11. Organizing skills for a team

12 minutes. Share skills across a team with naming, ownership, review, versioning and a catalog so people find and trust them.

One skill is easy to manage. Fifty skills written by ten people create overlap, drift and risk. The solution is the same discipline you use for code: ownership, review and a place to find things.

What you will learn

- Naming and scoping to avoid overlap
- Ownership, review and versioning
- A catalog that people can search
- Retiring and auditing skills

Naming and scope

Use a clear pattern such as `team-area-task`, and write descriptions that state when to use the skill and when not to. Overlapping descriptions cause the wrong skill to trigger. Merge duplicates rather than keeping variants.

Repository and review

```
skills/
  CODEOWNERS
  platform/
    triage-api-latency/
    rotate-certificates/
  data/
    backfill-checklist/
  catalog.md

git log --oneline -- skills/platform/triage-api-latency | head
./tools/check-skills.sh   # lint front matter, links and scripts
```

Keep skills in version control. Require a review from the owning team for changes, run a lint check for front matter and broken links, and test with a few sample prompts. Use tags or version numbers so a team can pin a known good version.

Catalog

Publish a catalog with each skill's name, description, owner, last review date and risk level. People search it before writing a new skill.

Retire and audit

Review skills on a schedule, such as every quarter. Remove skills nobody uses and flag skills that run scripts or touch production for a security review. Record who may install which skills.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Organizing skills for a team fits together

Part	What it does	What goes wrong
Naming and scope	Use a clear pattern such as team-area-task, and write descriptions that state when to use the skill and when not to.	No owner for a skill.
Repository and review	Keep skills in version control. Require a review from the owning team for changes, run a lint check for front matter and broken links, and test with a few sample prompts.	Skills with production access and no review.
Catalog	Publish a catalog with each skill's name, description, owner, last review date and risk level.	Duplicate skills with unclear differences.
Retire and audit	Review skills on a schedule, such as every quarter.	Never removing old skills.

Flow: Naming and scope to Repository and review / Repository and review to Catalog / Catalog to Retire and audit

Incident: Three skills for one job

Incident: *Overlapping skills made Claude choose inconsistently.*

```
skills: deploy-service, deploy-app, release-helper
all three describe deploying to production
rollback steps differ between them
```

Each team had written its own skill. Claude chose by description and sometimes picked one with an outdated rollback step. The platform team merged them into one with an owner, tests and a deprecation note on the others.

Common mistakes

- Duplicate skills with unclear differences
- No owner for a skill
- Skills with production access and no review
- Never removing old skills

Check yourself

What causes the wrong skill to trigger?

Answer: Overlapping descriptions

Who should review a production skill?

Answer: The owning team and security

Why version skills?

Answer: Teams can pin a known good version

What does a catalog show?

Answer: Owner, description and review date

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 12. Evaluating and regression testing skills

13 minutes. Build a small test set for a skill, check triggering and output quality, compare versions and catch regressions before release.

Changing one line in a skill can change when it triggers or how it behaves. Without tests you will not know. A small evaluation set makes changes safe and shows whether the skill adds value.

What you will learn

- Trigger tests: should and should not use the skill
- Output tests with clear grading
- Comparing with and without the skill
- Gating changes on results

Trigger tests

```
# evals/triggers.yaml
should_trigger:
  - "The checkout API p99 jumped after the deploy, help me triage"
  - "Latency is high on the orders service in prod"
should_not_trigger:
  - "Write a SQL query for monthly revenue"
  - "Explain what a Kubernetes pod is"
```

A good description triggers on the first group and stays quiet on the second. Include near misses, such as a database problem when the skill covers only API services.

Output tests

```
# evals/tasks.yaml
- id: latency-after-deploy
  prompt: "p99 doubled after the 14:10 deploy of checkout. Triage."
  must:
    - checks rollout history before proposing a change
    - uses read-only commands only
    - asks approval before rollback
  must_not:
    - runs kubectl rollout restart
```

Grade with a checklist, either by a person or by a model with a clear rubric you spot-check. Keep the rubric short, specific and tied to the skill's rules.

Compare and gate

Run each task with and without the skill, and between the old and new version. Record pass rate, tokens and time.

```
python evals/run.py --skill triage-api-latency --version HEAD --out results/head.json
python evals/compare.py results/main.json results/head.json --fail-on-regression
```

Fail the merge on any regression in the must or must-not checks. Add a test every time a real failure is found, so the suite grows from experience.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Evaluating and regression testing skills fits together

Part	What it does	What goes wrong
Trigger tests	A good description triggers on the first group and stays quiet on the second.	Only testing prompts that should trigger.
Output tests	Grade with a checklist, either by a person or by a model with a clear rubric you spot-check.	Not comparing against no skill.
Compare and gate	Run each task with and without the skill, and between the old and new version.	Fixing failures without adding tests.

Flow: Trigger tests to Output tests / Output tests to Compare and gate

Incident: A description edit that triggered everywhere

Incident: *A small wording change made a skill load for unrelated requests.*

```
prompt: "Write a SQL query for monthly revenue"
skill loaded: triage-api-latency
expected: no skill
```

Someone broadened the description to include performance in general. The skill then loaded for many unrelated requests. A negative trigger test failed in review and the description was narrowed again.

Common mistakes

- Only testing prompts that should trigger
- Grading outputs by feel
- Not comparing against no skill
- Fixing failures without adding tests

Check yourself

What is a negative trigger test?

Answer: A prompt that should not load the skill

Why compare with and without the skill?

Answer: To see whether it adds value

What gates a merge?

Answer: No regression in required checks

When add a new test?

Answer: After a real failure

MARGA by EknathaLabs marga.eknathalabs.com

Cheat sheet: Claude Skills

Skill folder layout, SKILL.md header, descriptions that trigger, structure, scripts, tests and a security review checklist.

Folder layout

```
incident-summary/
  SKILL.md
  template.md
  reference/format.md
  examples/outage-example.md
  scripts/check_summary.py
```

SKILL.md

```
---
name: incident-summary
description: Write a blameless incident summary from notes or a timeline. Use when the user asks f \
    or an incident summary, post-incident report or outage write-up.
---

Use template.md. Steps:
1. Collect start, end, impact, detection and mitigation.
2. Write "not stated" for anything the notes do not say.
3. Keep it under 300 words.
4. Run scripts/check_summary.py and fix what it reports.
```

> **Warning:** *The description decides whether the skill is ever used. Say what it does and when to use it, with real nouns and verbs. Check current name and length limits in the product documentation.*

Descriptions

```
Weak:   Helps with documents.
Better: Create and edit Word (.docx) files with headings, tables and tracked changes. Use when the \
     user asks for a Word document or to edit a .docx.
```

Add exclusions for neighbouring skills. Keep the key words first.

Progressive disclosure

```
Level 1: name and description (always)
Level 2: SKILL.md body (when triggered)
Level 3: linked files and scripts (when needed)
```

> **Warning:** *A giant SKILL.md wastes context. Move rarely needed detail to linked files, one level deep.*

Scripts

```
#!/usr/bin/env python3
"""Check a summary. Usage: python check_summary.py summary.md"""
import sys
text = open(sys.argv[1], encoding="utf-8").read()
problems = [] if len(text.split()) <= 300 else ["over 300 words"]
print("OK" if not problems else "\n".join(problems))
sys.exit(1 if problems else 0)
```

Use scripts for exact work. Keep output short and specific.

Test

```
Should trigger:     "write up last night's outage from these notes"
Should not trigger: "what does a 502 mean?"
```

Run 10 should-fire and 10 should-not requests, with and without the skill. Keep the task set in the repository.

Share safely

```
grep -rn "curl\|wget\|urlopen\|requests\.\|eval(\|exec(" skills/NAME/
```

> **Warning:** *A skill can steer Claude and include scripts that run. Read every file, look for network calls, downloads and shell commands, and run unknown skills in a sandbox first.*

MARGA by EknathaLabs marga.eknathalabs.com

Debug

```
ls -la .claude/skills/NAME/
head -6 .claude/skills/NAME/SKILL.md
```

Check location, exact file name SKILL.md, header syntax, then the description, then the body, then scripts.

> **Warning:** *Re-run your trigger and task tests after any model, product or description change.*

MARGA by EknathaLabs
marga.eknathalabs.com